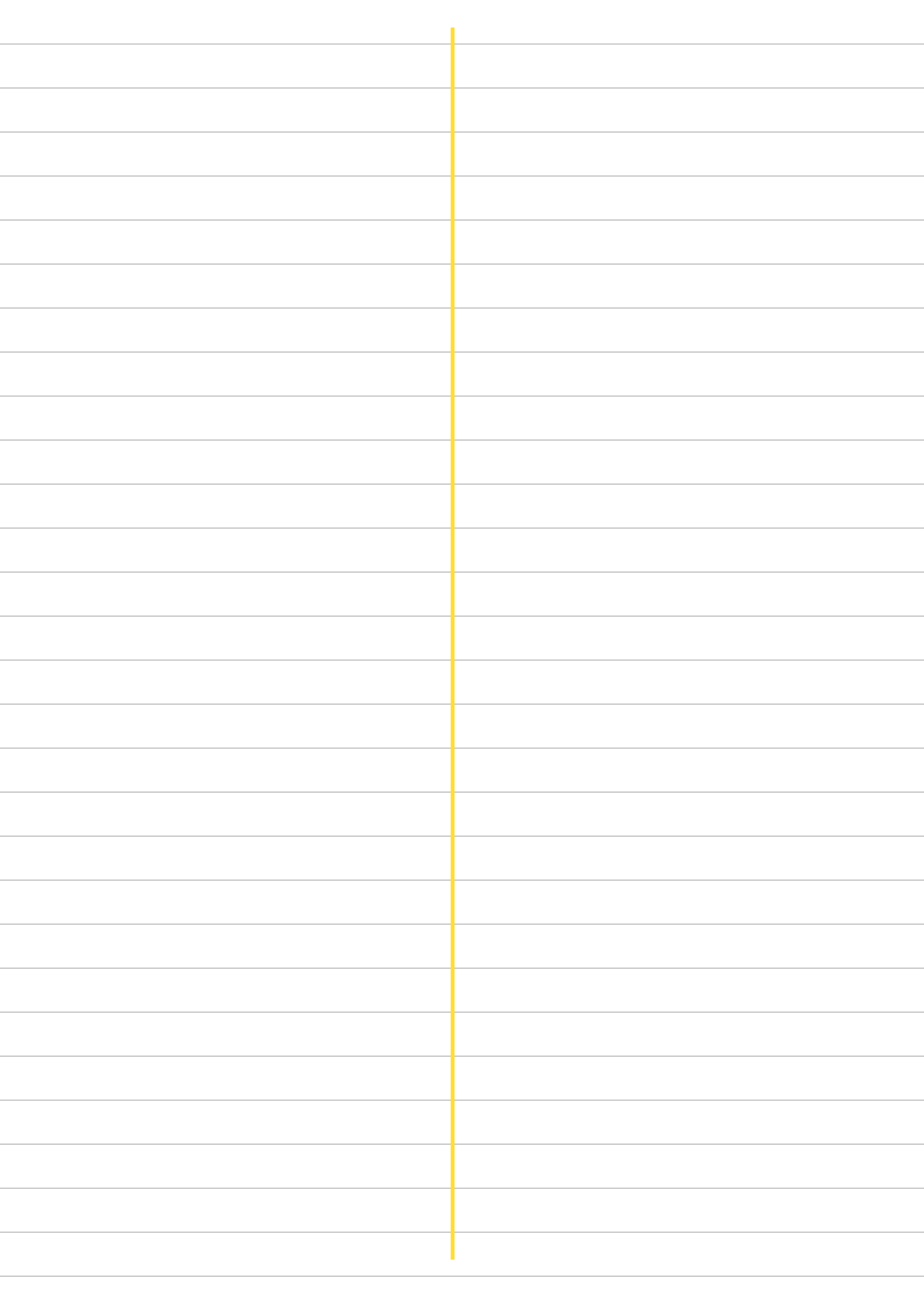




Bus organisation

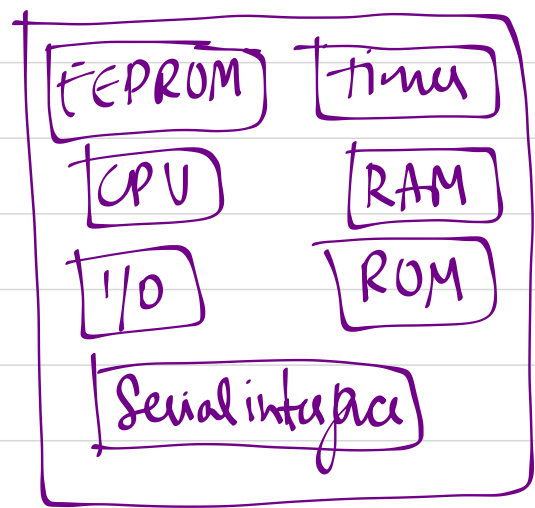
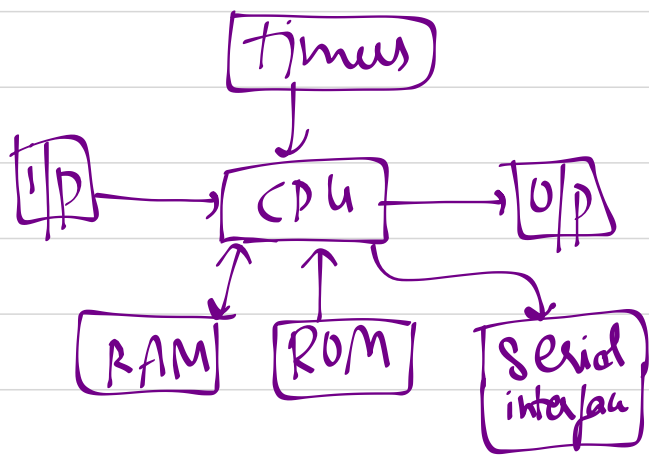
1m too
large to
write
refel ppt)

Microprocessors

- (1) CPU and several supporting chips
- (2) Silicon chip has only CPU capable of perf. logical & arithmetic op.
- (3) Dependent unit. It requires comb. of other chips like timer, interrupts, etc
- (4) doesn't contain built in I/O port
- (5) targeted for high end market where perf. is imp.
- (6) Most of the time general purpose in design & oper.
- (7) limited power saving options
- (8) 8086

Microcontrollers

- (1) CPU and other peripherals on a single chip
- (2) highly integrated chip that contains CPU, RAM, ROM, timer and interrupts CU & dedicated I/O ports
- (3) self contained unit and doesn't req. external timer, interrupt, etc.
- (4) contains multiple built in I/O ports
- (5) Targeted for emb. market where perf. is not so critical
- (6) App. oriented or domain specific
- (7) lot of power saving features
- (8) STM32



ARM → Advanced RISC machine

↳ microprocessors

→ doesn't make microcontrollers

→ designs processors and give licenses of processor designs to various mc. vendors.

→ these designs are called "Intellectual property" (IP) and the business model is called IP licensing.

Microcontroller memories

3 types →

(1) Prog. mem. (non volatile)
ROM, flash mem.

↓
data not lost when power shuts down.

(2) data mem. (volatile)

↓
→ stores variables during prog. execution
→ very fast.

→ flash mem. is similar to RAM (read, write & overwrite) but is non-volatile.

RISC

- (1) Reduced inst. set comp. processor
- (2) fixed inst. size
- (3) less addressing modes
- (4) more regs
- (5) easier to understand
- (6) fetch time / execution time is same for all inst.
- (7) eg. ARM
- (8) easier design of pipelining
- (9) orthogonal inst. set
- (10) reg-to-reg.
- (11) code density is less

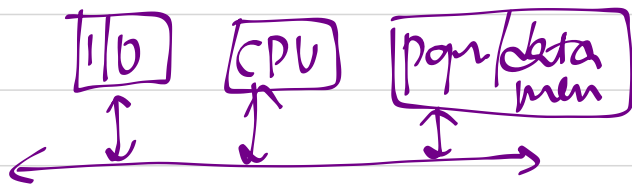
CISC

- (1) complex inst. set comp. processor.
- (2) variable inst. size
- (3) more
- (4) less
- (5) complex
- (6) vary w.r.t. inst.
- (7) eg. 8086 μ p
- (8) complex
- (9) non-orthogonal
- (10) reg-to-mem.
- (11) code ρ is more

Von Neumann arch (aka Princeton arch)

(1) addr space is shared
b/w prog. mem. & data mem.
ROM RAM

(2) single shared bus
for inst. and data
fetching



(3) speed of execution is less

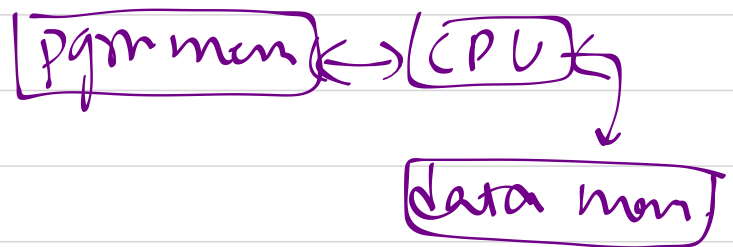
(4) complexity of design
is less

(5) sequential processing

Harvard architecture

(1) not shared

(2) separate buses for
inst. & data fetching



(3) speed ↑

(4) processor design is
complex

(5) parallel processing

Endianness

Little 0x12345678

0x100	0x12
0x101	0x34
0x102	0x56
0x103	0x78

big

0x100	0x78
0x101	0x56
0x102	0x34
0x103	0x12

Floating pt arithmetic

used to rep. very large
and very small no.s
precisely

$$(1163.125)_{10}$$

$$2 \overline{) 1163}$$

$$2 \overline{) 581} - 1$$

$$2 \overline{) 290} - 1$$

$$2 \overline{) 145} - 0$$

$$2 \overline{) 72} - 1$$

$$2 \overline{) 36} - 0$$

$$2 \overline{) 18} - 0$$

$$2 \overline{) 9} - 0$$

$$2 \overline{) 4} - 1$$

$$2 \overline{) 2} - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$2 \overline{) 137}$$

$$2 \overline{) 68} - 1$$

$$2 \overline{) 34} - 0$$

$$2 \overline{) 17} - 0$$

$$2 \overline{) 8} - 1$$

$$2 \overline{) 4} - 0$$

$$2 \overline{) 2} - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

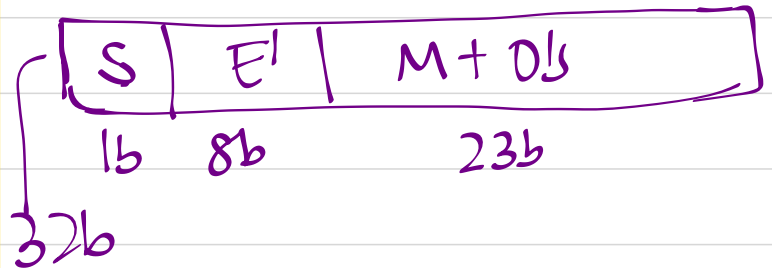
$$1 - 0$$

$$1 - 0$$

$$1 - 0$$

$$E' = 10001001$$

$$\text{Mantissa} = 0010001011001$$



double precision.

$$E' = 1023 + 10 = 1033$$

$$2 \overline{) 1033}$$

$$2 \overline{) 516} - 1$$

$$2 \overline{) 258} - 0$$

$$2 \overline{) 129} - 0$$

$$2 \overline{) 64} - 1$$

$$2 \overline{) 32} - 0$$

$$2 \overline{) 16} - 0$$

$$2 \overline{) 8} - 0$$

$$1 - 0$$

$$2 \overline{) 8}$$

$$2 \overline{) 4} - 0$$

$$2 \overline{) 2} - 0$$

$$1 - 0$$

$$\text{normalisation: } 1.0010001011001 \times 2^{10}$$

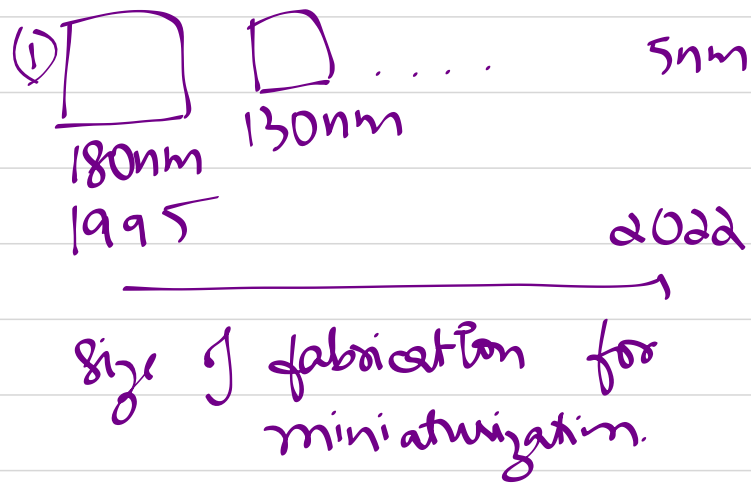
$$S = 0, E = 10$$

single precision:

Biased comp:

$$E' = 127 + 10 = 137$$

Evolution of ARM processor!



→ The first ARM processor (ARM1) was developed at Acorn comp. limited by a team led by sofie wilson & Steve furber.

Joint venture b/w Acorn comp., apple, VLSI tech. Acorn provided 12 emp, VLSI tools & apple \$3M invest.

→ ARM processor uses RISC architecture
ARM company doesn't make diff. applications
ARM company makes CPU cores

ARM company provides CPU core to diff. companies.

diff. vendors use ARM CPU core and develop diff. app. acc. to their needs

(1) ARM7TDMI → thumb → 16bit inst. set
ARMV4 → 32bit inst. set

a) Arm state : for higher category of app.
b) thumb state : for lower category of app.

(2) ARM9 → added sm. more inst. set features like execution of java inst.

(3) ARM11 →

thumb2
SIMD
thumb

ARMV6

thumb2 - intermixing of 16bit & 32bit inst. set. Intermixed inst. can be done.

(4) ARM cortex fam

A: designed for high perf. open app. prog.
R: for high end real time emb. sys.
M: for microcontroller based emb. sys.

Evolution

ARM inst. (32 bit)

↓
ARM inst + thumb inst

32b | ARM → 16b
ARMV6T → thumb

ARM inst. + thumb inst

32b | 16b
[DSP] ARMV6TE
thumb w/
extension

ARM inst. 32b + thumb inst.

[floating pt] 16b
[DSP, SIMD] ARM11
ARMV6

ARM inst. 32b + thumb-2

[floating pt] (16b + 32b)
[DSP, SIMD] ARM cortex
ARM7 [floating pt]
[DSP, SIMD]

U2

→ Cortex-M-processor.

→ 32-bit architecture



Regs, datapath, bus width
can process 32 bits of
data at once
(faster computation)

→ Thumb ISA

↳ thumb / thumb-2

↳ mix of:

16-bit inst: smaller code size

32-bit inst: higher perf.

→ Adv: - Saves memory

- still maintains perf.

→ Advantages:

→ Low power:

- designed relatively smaller
than other 32-bit designs.

- very low power consumption
($< 100 \mu A / MHz$)

- has deep sleep modes.

→ performance:

- executes complex & demanding
applications

→ Energy efficiency

- combining low power &
high perf. char. gives
excellent energy eff.

→ code density

- Thumb ISA provides exc.
code density → reduces
power cons., cost & size

→ Interrupts:

- supports 240 vectored
interrupts and multiple
levels of interrupt priorities.
(from 8 to 256 levels)

- makes it suitable for real
time control applications.

→ Easy prog. using 'C' lang.

- has simpler linear mem.
mapping. and no architec-
restrictions.

→ Easy scalability

- easy scaling from low cost
simple designs to high cost
multi processor designs.

→ Debug features

- allows user to analyse design prob. easily.
- provides halt, single stepping, trace to capture prog. flow, data changes, profiling info, etc.

→ OS support.

- features make OS implementation easier & OS operations more efficient.

→ versatile sys. features.

- Bit addressable mem. range (Bit band feature) and MPU (Mem. protection unit)

→ Software portability & reusability:

- CMSIS (Cortex microcontroller software interface std) provides std. header files and APIs making programming easier.

→ choices (Devices, tools, OS, etc.)

- no. of choices are a lot.

→ Features of M3/M4 processor.

- 3 stage pipeline design (fetch - decode - execute.)

- Harvard architecture w/ unified memory space (same mem. address).
faster access (parallel fetch + data access)

- 32 bit addressing can address $2^{32} = 4GB$ mem.

→ AMBA bus

- on chip interfacing allows higher throughput.

- NVIC (Nested vectored interrupt controller)
↓
up to 240 interrupts

→ Having MPU sys.

- supports bit band mem. mapping.

Programmers Model

Cortex M4 has

- 2 operating STATES
- 2 operating MODES
- 2 access LEVELS

- State: what type of execution?
(normal/debug)
- Mode: what kind of code is running
(ISR/normal)
- access level: what permissions you have?

(1) operating STATES.

- Thumb State (default) (D)
 - executes prog. inst.
 - only supports thumb inst.
(no ARM mode)
 - prog. runs normally.

- Debug state
 - entered when processor is halted (by debugger / after hitting a breakpoint)
 - CPU stops execution & lets you inspect regs, mem.

(2) Operation Modes:

(only inside thumb state)

- (D) → Thread mode: (normal prog.)
 - executes application code [main()]
 - can run in privileged & unprivileged access levels.

- Handler mode (Interrupt mode)
 - executes exception handlers such as ISR (interrupt service routines)
 - always runs in privileged mode ONLY.

when interrupt occurs
thread mode → handler mode

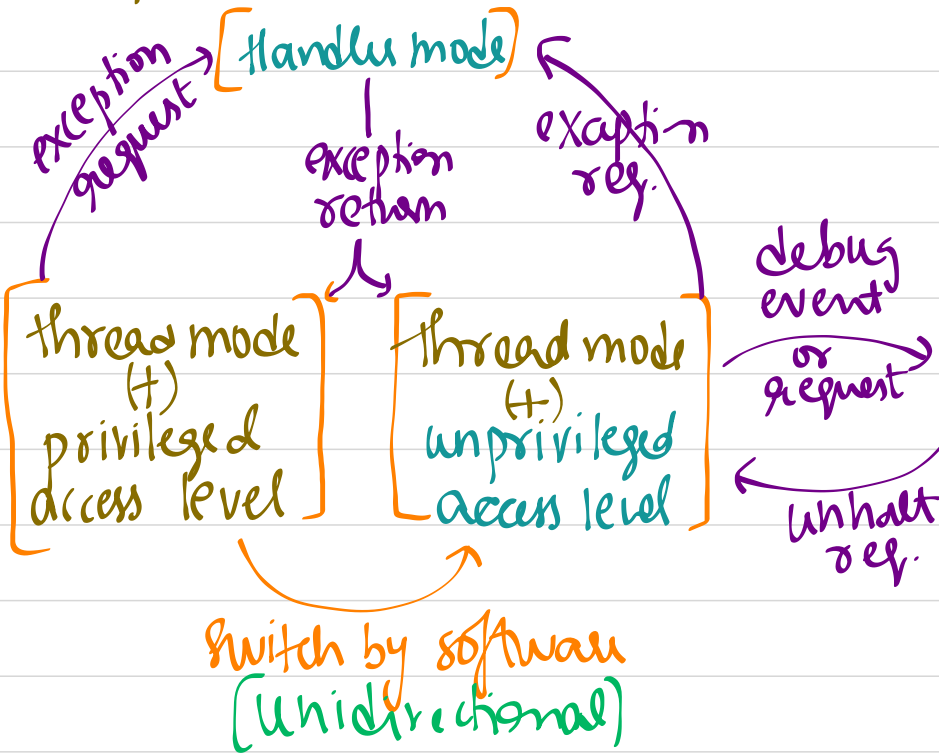
after ISR completes
handler → thread

(3) Access levels

- Privileged (default)
 - full control over sys.
 - can access all mem. & regs.
- Unprivileged
 - restricted access
 - cannot access protected mem. or modify sys. ctrl. regs.

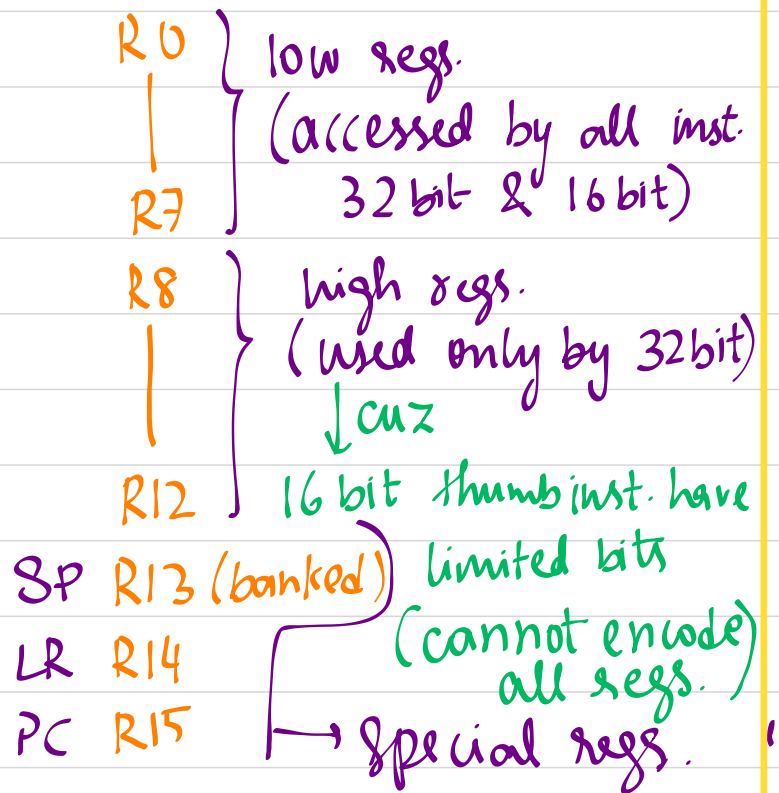
thumb state

→ default in yellow, others in blue



* debug operation only possible when debugger connected.

Register bank.



→ ARM → load-store architecture
 mem. → reg. → process → reg. → mem.
 (CANNOT) mem. → ALU

→ Stack - temp. storage in RAM

→ LIFO (last in first out)

→ Stack pointer (SP / R13)

— points to top of stack.

→ PUSH — store data in stack.

— SP ↓

→ POP — retrieve data (SP ↑)

→ 2 stack pointers:

→ MSP (main stack pointer)

— default after reset

— used in handler mode (always)

→ PSP (process stack pointer)

— used in thread mode (OS sys)

* Selection is controlled by CONTROL reg.

→ Stack inst.

→ PUSH / POP

↳ always 32 bit
↳ address must be 4 byte aligned.

STM (Store multiple) PUSH

LDM (Load multiple) POP

→ Link Register (LR/R14)

- stores return address during function call.

→ Program counter

- holds address of next inst.

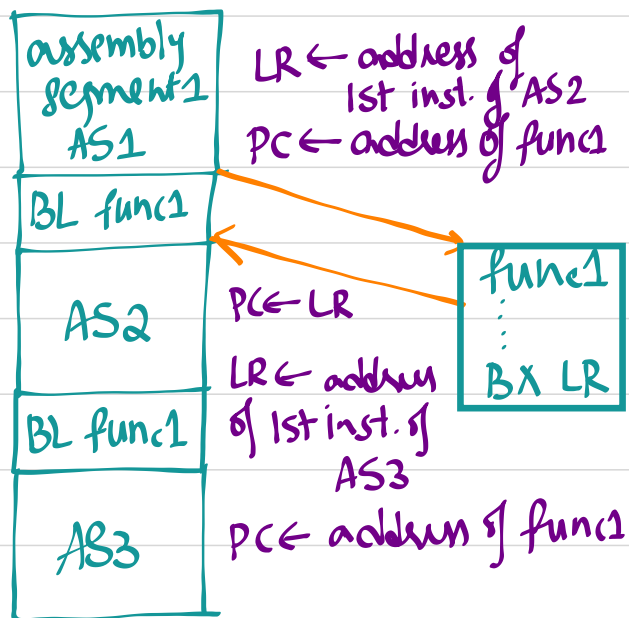
→ When calling func,

LR = return address

→ return from func.

PC = LR (using BX LR)

eg.



→ Special regs.

→ Program status reg (xPSR)

→ APSR → Application PSR (flags)

EPSR → Execution PSR

I PSR → Interrupt PSR

Access inst.

eg. MRS r0, PSR ; read combined prog. status word

MSR PSR, r0 ; write

→ APSR flags

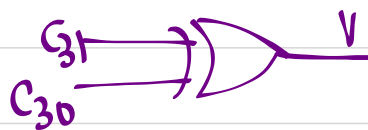
→ N (negative) : 1 if result is (-ve)

→ Z (zero) : 1 if result is 0

→ C (carry) : 1 if carry generated (add)
↳ for 31st bit.

for subⁿ: inverse of borrow o/p status.

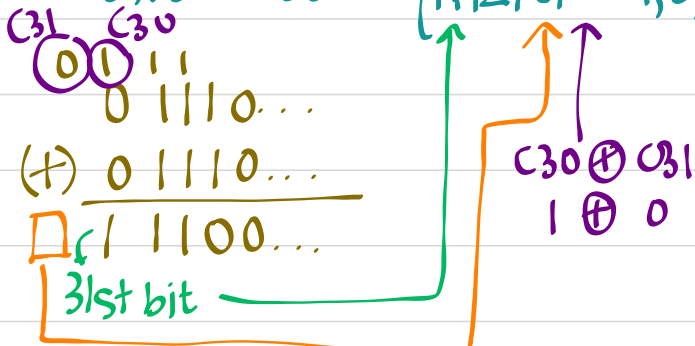
→ V (overflow) : signed overflow



→ S (saturation) : used in DSP

eg. 0x70000000 + 0x70000000

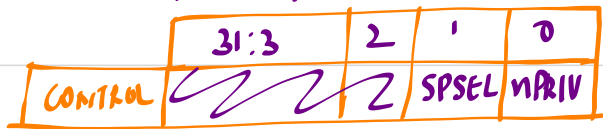
= 0xE0000000 [N, Z, C, V = 1, 0, 0, 1]



→ Control reg.

→ defines

- selection of stack pointer (MSP / PSP)
- access level in thread mode (privileged / unprivileged)



→ nPRIV (bit 0)

- defines privileged level in thread mode.
- nPRIV = 0 [default] - privileged
= 1 - unprivileged

→ SPSEL (bit 1)

- defines stack pointer selection.
- SPSEL = 0 [default] → MSP
= 1 → PSP
in thread mode

→ in handler mode, SPSEL = 0 (always)

Q) nPRIV = 1 thread mode, how to change to privileged?
→ cannot do it directly

→ thread (unpriv.)

↓
Exception (SVC) interrupt

↓
handle mode (privileged) ^(always)
by default

↓
modify CONTROL (nPRIV = 0)

↓
return

↓
thread mode (privileged)

Data Definition Derivatives

→ used to store data in memory (not inst.)

Basic Derivatives:

→ DCB (8bit) — Byte 0x12
DCW (16bit) — half word 0x1234
DCD (32bit) — word 0x1234 5678
DQA (64bit) — double word
0x1234 5678 FFAB CDEF

Floating point

DCFS (32bit float) 1.234
DCFD (64 bit double) 1.341956

eg: name1 DCD 0x1234 5678

Strings

eg: name DCB "Hello\n", 0
↑
null termination

→ equivalent 'c' mapping

DCB — char (signed)
DCW — short (signed)
DCD — int/long (signed)
DQA — long long (signed)
DCFS — float
DCFD — double / long double

Instructions:

used for 8bit imm. value

→ MOV } copy value
MOVS } copy + update flag
MOVW lower 16 bits
MOVT upper 16 bits
MVN move neg. value
MRS Read special reg.
MSR write special reg.

Instruction	Destination	Source	Operation
MOV	R4,	R0	Copy value from R0 to R4
MOVS	R4,	R0	Copy value from R0 to R4 with APSR update
MRS	R7,	PRIMASK	Copy value of PRIMASK to R7
MSR	CONTROL,	R2	Copy value of R2 into CONTROL
MOV	R3,	#0x34	Set R3 value to 0x34
MOVS	R3,	#0x34	Set R3 value to 0x34 with APSR update
MOVW	R6,	#0x1234	Set R6 to a 16-bit constant 0x1234
MOVT	R6,	#0x8765	Set the upper 16-bit of R6 to 0x8765
MVN	R3,	R7	Move negative value of R7 into R3

- Identify the operation of following code snippets:

Listing 1:

```
MOVS R0, #0x12      ; Set R0 = 0x12 (hexadecimal)
MOV R1, #'A'        ; Set R1 = ASCII character A
```

Listing 2:

```
NVIC_IRQ_SETEN EQU 0xE000E100
NVIC_IRQ0_ENABLE EQU 0x1
LDR R0, =NVIC_IRQ_SETEN      ; Put 0xE000E100 into R0
MOV R1, #NVIC_IRQ0_ENABLE    ; Put immediate data (0x1) into
STR R1, [R0]                  ; Store 0x1 to 0xE000E100,
```

equates <name EQU value> syntax

load → *address*

move → *value*

store → *address*

[address] ⇒ store into that address

- Identify the operation of following code snippet:

Listing 3:

```
LDR R3, =MY_NUMBER      ; Get the memory location of MY_NUMBER
LDR R4, [R3]             ; Read the value 0x12345678 into R4
LDR R0, =HELLO_TEXT      ; Get the starting address of HELLO_TEXT
BL PrintText             ; Call a function called PrintText to display string
MY_NUMBER DCD 0x12345678
HELLO_TEXT DCB "Hello\n", 0 ; Null terminated string
```

→ address of my-no.

→ value stored in R3

→ address of HELLO_TEXT

→ Call a function called PrintText to display string

Branch w/ link - used to call a func.

MY_NUMBER DCD 0x12345678

HELLO_TEXT DCB "Hello\n", 0 ; Null terminated string

→ R0 = address of "Hello\n"

LR = return address

PC = PrintText (func)

prints the string pointed by R0.

Inside PrintText

BX LR ⇒ PC = LR

so it can return back to caller.

Memory Access inst.

Data type	Load(Read from memory)	Store(Write to memory)
8 bit unsigned	LDRB(Zero extend)	STRB
8 bit signed	LDRSB(Sign extend)	STRB
16 bit unsigned	LDRH(Zero Extend)	STRH
16 bit signed	LDRSH(Sign Extend)	STRH
32 bit	LDR	STR
Multiple 32 bit	LDM	STM
Double word(64 bit)	LDRD	STRD

load: mem. $\rightarrow$ reg.

store: reg. $\rightarrow$ mem.

Ex $\rightarrow$

mem[0x20000000] = 0x83 (1)

R1 = 0x20000000 0x7F (2)

R0 = 0xFFFFFFFF 0000 1111

O/P

LDRB R0, [R1] R0 = 0x00000083 (1)

= 0x0000007F (2)

LDRSB R0, [R1] R0 = 0xFFFFFFFF83 (1)

$\rightarrow$ MSB is taken = 0x0000007F (2)

mem[addr] = 0x9083 (1)

R1 = addr 0x7FFF (2)

LDRH R0, [R1] R0 = 0x00009083 (1)

= 0x00007FFF (2)

LDRSH R0, [R1] R0 = 0xFFFF9083 (1)

0x00007FFF (2)

mem = 0x12345678

LDR R0, [R1] R0 = 0x12345678

R0 = 0x12345678

STRB R0, [R1] mem = 0x78

STRH R0, [R1] = 0x5678

STR R0, [R1] = 0x12345678

mem[0x20000000] = 0x11111111

[0x20000004] = 0x22222222

R1 = 0x20000000

LDM R1, {R0, R2} R0 = 0x11111111

R2 = 0x22222222

R0 = 0xAAAAAAAA1

R2 = 0xBBBBBBBB2

STM R1, {R0, R2} M[R1] = 0xAAAAAAAA1

M[R1+4] = 0xBBBBBBBB2

M[R2] = 0x11111111

M[R2+4] = 0x22222222

LDRD R0, R1, [R2] R0 = 0x11111111

R1 = 0x22222222

R0 = 0xAAAAAAAA1

R1 = 0xBBBBBBBB2

STRD R0, R1, [R2] M[R2] = R0

M[R2+4] = R1

Pre Index	Description
LDRB Rd,[Rn,#offset]	Read byte from memory location Rn+offset
LDRSB Rd,[Rn,#offset]	Read byte and sign extend from memory location Rn+offset
LDRH Rd,[Rn,#offset]	Read halfword from memory location Rn+offset
LDRSH Rd,[Rn,#offset]	Read halfword and sign extend from memory location Rn+offset
LDR Rd,[Rn,#offset]	Read word from memory location Rn+offset
LDRD Rd1,Rd2,[Rn,#offset]	Read double-word from memory location Rn + offset
STRB Rd,[Rn,#offset]	Store byte to memory location Rn+offset
STRH Rd,[Rn,#offset]	Store Halfword to memory location Rn+offset
STR Rd,[Rn,#offset]	Store word to memory location Rn+offset
STRD Rd1,Rd2,[Rn,#offset]	Store double word to memory location Rn+offset

$$m[0x20000004] = 0x83 \quad C1$$

$$R1 = 0x20000000 \quad 0x73 \quad C2$$

$$R0 = 0xAAAAAAAA \quad (C1)$$

$$LDRB R0, [R1, \#4] \quad R0 = 0x00000083 \\ = 0x00000073 \quad C2$$

$$LDRSB R0, [R1, \#4] \quad R0 = 0xFFFFFFFF83 \\ = 0x00000073 \quad C2$$

$$\left. \begin{array}{l} LDRH R0, [R1, \#4] \\ LDRSH R0, [R1, \#4] \end{array} \right\} 16bit$$

$$LDR R0, [R1, \#4] \quad - 32bit$$

$$m[0x20000000] = 0x83$$

$$m[0x20000004] = 0xFF$$

$$R1 = 0x20000004$$

$$R2 = 0xFFFFFFFF00$$

$$R3 = 0x3457AABB$$

$$LDRD R2, R3, [R1, \#-4]$$

$$R2 = 0x00000083$$

$$R3 = 0x000000FF$$

$$R0 = 0x12345678$$

$$R1 = 0x20000000$$

$$STRB R0, [R1, \#4] \quad m[R1+4] = 0x78$$

$$m[20000004] = 0x78$$

$$STRH R0, [R1, \#4] \quad m[R1+4] = 0x5678$$

$$STR R0, [R1, \#4] \quad m[R1+4] = 0x12345678$$

$$m[0x20000000] = 0xAAAAAAAA$$

$$m[0x20000004] = 0xBBBBBBBB$$

$$R1 = 0x20000008$$

$$STRD R2, R3 [R1, \#-4]$$

$$m[R1-4] = R2$$

$$m[R1-4+4] = m[R1] = R3$$

Instruction	Operation
LDR R0, [R1, #0x8]!	R0= Read from memory[R1+8] R1= R1+8
STRD R2,R3, [R1,#-8]!	Store R2,R3 at memory [R1-8] R1= R1-8
Instruction	Operation
LDR R5, [R0,R7]	R5= Read from memory[R0+R7]
STR R2, [R1,R4]!	Store R2 at memory [R1+R4] R1= R1+R4
LDR r0,[r1,r2,LSL#2]!	r0=Read from memory[r1+(r2<<2)] r1=r1+(r2<<2)
LDR r0,[r1,r2,LSL #0x2]	r0=Read from memory[r1+(r2<<2)]

(!) → specifies whether the reg. holding the address should be updated (write back) when the inst. is completed.

only LSL allowed
LSL → logical left shift
shift range is 0 to 3

Note

Pre Index	Description
LDRB Rd,[Rn],#offset	Rd= Read byte at [Rn], Rn=Rn+offset
LDRSB Rd,[Rn],#offset	Rd=sign extended byte at [Rn], Rn=Rn+offset
LDRH Rd,[Rn],#offset	Rd= Read halfword at [Rn], Rn=Rn+offset
LDRSH Rd,[Rn],#offset	Rd=sign extended word at [Rn], Rn=Rn+offset
LDR Rd,[Rn],#offset	Rd= Read word at [Rn], Rn=Rn+offset
LDRD Rd1,Rd2[Rn],#offset	Rd= Read double word at [Rn], Rn=Rn+offset
STRB Rd,[Rn],#offset	[Rn]=LSB from Rd, Rn=Rn+offset
STRH Rd,[Rn],#offset	[Rn]=Lower 16 bits from Rd, Rn=Rn+offset
STR Rd,[Rn],#offset	[Rn]=store Rd, Rn=Rn+offset
STRD Rd1,Rd2,[Rn],#offset	[Rn]=double word from Rd1,Rd2, Rn=Rn+offset

$R1$
 $m[0x20000000] = 0x12345678$
 $LDR R0, [R1], \#4 \quad R0 = m(R1)$
 $R1 = 0x20000004$

$R1 = 0x20000008$

$R2 = 0xAAAAAAAA$

$R3 = 0xBBBBBBBB$

$STRD R2, R3, [R1], \#-8$
 $m(R1) = m(0x20000008) = R2$
 $m(R1+4) = m(0x2000000C) = R3$
 $R1 = R1 - 8 = 0x20000000$

Instruction	Operation
LDmia Rn,{reg list}	Read multiple words from memory location specified by Rn Address Increment After (IA) each read.
LDmdb Rn,{reg list}	Read multiple words from memory location specified by Rn. Address Decrement Before (DB) each read.
STmia Rn,{reg list}	Write multiple words to memory location specified by Rn. Address increment after each write.
STmdb Rn,{reg list}	Write multiple words to memory location specified by Rn. Address Decrement Before each write.

$m(0x2000-0000) = 0x83$
 $m(0x2000-0004) = 0x1000-00FF$
 $m(8) = 0xFFFF-00FF$
 $m(C) = 0x8900-0011$
 $R0 \text{ to } R4 = 0x0000-0000$

$LDR R4, =0x2000-0000$
 $LDmia R4, \{R0-R3\}$ *first reg. is R0 (lowest reg.)*
 $R0 = m(R4) = 0x83$
 $R1 = m(R4+4) = 0x1000-00FF$
 $R2 = m(R4+8)$ *initial pointer*
 $R3 = m(R4+12)$ *points to bottom of data*
 $R4 = 0x2000-0000$

$LDR R4, =0x2000-0010$ *16 C-12 10-16*

$LDmdb R4!, \{R0-R3\}$

$R4 = R4 - 4 = 0x2000-000C$

$R3 = m(R4) = 0x8900-0011$

$R2 = m(R4-4) = 0xFFFF-00FF$

$R1 = m(R4-8) = 0x1000-00FF$

$R0 = m(R4-12) = 0x83$

$R4 = 0x2000-0000$

first reg. is R3 (highest reg.)
Las

initial pointer points to the top (limit) of data

Questions

1) Identify the operation of following instructions:

- MOV R2,R3, LSL #4
- MOV R2,R3,ASR R1
- MOV R2,R3, LSR R1

2) Write a program segment to transfer 32 bit data starting at 0x2000 0010 to a location 0x2000 0024.

```
LDR R0,= 0x2000 0010
LDR R1,= 0x2000 0024
LDR R2,[R0]
STR R2,[R1]
```

47

MGR,ECE,RVCE

3) Write the code snippet to access the contents of the array **myDat** and store in **newDat** declared as follows:

```
AREA |.data|,DATA,READWRITE,ALIGN=3
myDat DCW 1,2,3,4,5,6,
newDat SPACE 12

MOV R4,#6
MOV R3,#0
LDR R0,= myDat
LDR R1,= newDat

repeat
    LDRH R2, [R0,R3,LSL #1]
    STRH R2,[R1, R3,LSL #1]
    ADD R3,R3,#1
    SUBS R4,R4,#1
    BNE repeat
```

define const. have word (2 bytes) 16bit LDRH STRH

- $R2 = R3 \ll 4$ logical left
 - $R2 = R3 \ggg R1$ arithmetic right
 - $R2 = R3 \gg R1$ logical right
- $m[R0] \rightarrow m[R1]$

```
2) LDR R0,=0x0000_0010
LDR R1,=0x0000_0024
LDR R2,[R0] R2 = m(R0)
STR R2,[R1] m(R1) = R2
```

```
3) MOV R4,#6 loop counter set to 6.
MOV R3,#0
LDR R0,=myDat
LDR R1,=newDat
```

```
repeat
    LDRH R2, [R0,R3,LSL #1]
    STRH R2, [R1,R3,LSL #1]
    ADD R3,R3,#1 increment R3
    SUBS R4,R4,#1 decrement R4
    BNE repeat
    ↳ branch not equal
    if R4 ≠ 0 → loop to "repeat"
```

$m[2000] \rightarrow m[2002] \dots +12$

$R0 + (R3 \times 2)$

Mem. add^y = base + (index X data size)
in bytes

$R0 + R3 \times 2 \text{ (LSL \#1)}$
 $\times 4 \text{ (LSL \#2)}$

Questions

- Identify the difference between following instructions:

i. LDRGT r0, [r1, #4]!

ii. LDRGT r0, [r1, #4]

iii. LDRGT r0, [r1], #4

- Write the program segment to transfer 32 bit data starting at 0x2000 0010 to a location 0x2000 0024.

LDR R0, = 0x2000 0010

;LDR R1, = 0x2000 0024

LDR R2, [R0], #0x14

STR R2, [R0]

49

MGR,ECE,RVCE

- Write the code snippet to access the contents of the array **myDat** and store in **newDat** declared as follows: (use post-index)

AREA |.data|, DATA, READWRITE, ALIGN=3

myDat DCH 1,2,3,4,5,6,

newDat SPACE 12

MOV R4, #6

LDR R0, = myDat

LDR R1, = newDat

repeat

LDRH R2, [R0], #2

STRH R2, [R1], #2

SUBS R4, R4, #1

BNE repeat

1) i) $r0 = m[r1 + 4]$

$r1 = r1 + 4$

ii) $r0 = m[r1 + 4]$

$r1 = r1$

iii) $r0 = m[r1]$

$r1 = r1 + 4$

GT
(greater than)
→ executes only if
Z=0
N=V

Note

GT (>) : N=V & Z=0

LT (<) : N!=V

GE (>=) : N=V

LE (<=) : N!=V & Z=1

2) LDR R0, = 0x2000-0010

LDR R1, = 0x2000-0024

LDR R2, [R0], #0x14

$R0 = R0 + 14$

STR R2, [R0]

3)

MOV R4, #6 loop counter = 6

LDR R0, = myDat

LDR R1, = newDat

repeat

LDRH R2, [R0], #2

STRH R2, [R1], #2

SUBS R4, R4, #1 decrement R4 by 1

BNE repeat

↳ loops as long as $R4 \neq 0$

↳ branch not equal to 0

⇒ $R4 \neq 0 \rightarrow$ go to "repeat"

Questions

Multiple loads and stores...

- 1) What is the content of r0 after execution of following instruction and identify the locations where registers are stored?

STMDB r0!, {r1-r3}

r0 = r0 - 3x4

r3 = r0 - 4 r2 = r0 - 8 r1 = r0 - 12

- 2) Write an assembly instruction to load 4 words starting at memory location 0x80000000 from registers r0-r3? (Assume r9 contains the base address 0x80000000)

LDMIA r9, {r0, r1, r2, r3} or

LDMIA r9, {r0-r3}

- 3) Can the LDM instruction be conditional in ARM?

Yes

Z == 1

LDMEQIA r0!, {r1-r3} (Zero flag = 1)

- 4) How many registers can be loaded or stored using a single LDM or STM instruction?

All general purpose registers

STMDB R0, {R0-R12}

1) **STMDB**
↳ Starts from r3

r3 = r0 - 4 ; r0 = r0 - 12
r2 = r0 - 8 = r0 - 3x4
r1 = r0 - 12

2) **LDMIA r9, {r0-r3}**

3) **Note**
{EQ(=) : Z == 1}

4) **all GP regs.**
LDMIA R0, {R0-R12}

Instruction	Operation	
ADD Rd, Rn, Rm	$Rd = Rn + Rm$	'W' indicate wide to generate machine code of 32 bit with 12 bit immediate data. Some assemblers with 8 bit immediate data generate 16 bit machine code. No difference in armasm in Keil. It always generate 32 bit machine code
ADD Rd, Rn, #immed	$Rd = Rn + \#immed$	
ADC Rd, Rn, Rm	$Rd = Rn + Rm + \text{carry}$	
ADC Rd, #immed	$Rd = Rd + \#immed + \text{carry}$	
ADDW Rd, Rn, #immed	$Rd = Rn + \#immed$	
SUB Rd, Rn, Rm	$Rd = Rn - Rm$	
SUB Rd, #immed	$Rd = Rd - \#immed$	*No flags updated because suffix 'S' is not there * All instructions are conditional
SUB Rd, Rn, #immed	$Rd = Rn - \#immed$	
SBC Rd, Rn, #immed	$Rd = Rn - \#immed - \text{borrow}$	
SBC Rd, Rn, Rm	$Rd = Rn - Rm$	

MGR, ECCE, RVCE

*No flags updated because suffix 'S' is not there
 * All instructions are conditional

ADD R0, R1, R2 $R0 = R1 + R2$

ADD R3, R4, #0x15 $R0 = R1 + 21$
 $0001\text{ }0101 = 16 + 4 + 1 = 21$

ADC R0, R1, R2 $R0 = R1 + R2 + C$
 carry flag.

ADC R5, #0x10 $R0 = R1 + 16 + C$

ADDW R0, R1, #0xABC
 $R0 = R1 + 2748$

32 bit wide inst. allowing
 12 bit immediate data

SUB R0, R1, R2 $R0 = R1 - R2$

SUB R3, #0x05 $R3 = R3 - 5$

SUB R4, R5, #0xFF $R4 = R5 - 255$

SBC R0, R1, #0x01 $R0 = R1 - 1 - B$
 in ARM { carry flag acts as "NOT" Borrow } borrow

SBC R6, R7, R8 $R6 = R7 - R8 - B$

Instruction	Operation
RSB Rd, Rn, #immed	$Rd = \#immed - Rn$ (reverse subtraction)
RSB Rd, Rn, Rm	$Rd = Rm - Rn$
MUL Rd, Rn, Rm	$Rd = Rn * Rm$
MLA Rd, Rn, Rm, Ra	$Rd = Ra + Rn * Rm$ (Multiply & accumulate)
MLS Rd, Rn, Rm, Ra	$Rd = Ra - Rn * Rm$ (Multiply & subtract)
SMULL RdLo, RdHi, Rn, Rm	$\{RdHi, RdLo\} = Rn * Rm$ (Signed multiply long)
SMLAL RdLo, RdHi, Rn, Rm	$\{RdHi, RdLo\} += Rn * Rm$ (Signed MLA long)
UMULL RdLo, RdHi, Rn, Rm	$\{RdHi, RdLo\} = Rn * Rm$ (Unsigned MUL long)
UMLAL RdLo, RdHi, Rn, Rm	$\{RdHi, RdLo\} += Rn * Rm$ (Unsigned MLA long)

* Only RSB is made conditional updating of flags

* All instructions are conditional

RSB R0, R1, #100 $R0 = 100 - R1$

RSB R0, R1, R2 $R0 = R2 - R1$

MUL R0, R1, R2 $R0 = R1 * R2$

MLA R0, R1, R2, R3 $R0 = R3 + (R1 * R2)$

MLS R0, R1, R2, R3 $R0 = R3 - (R1 * R2)$

SMULL R0, R1, R2, R3 $\{R1, R0\} = R2 * R3$

SMLAL R0, R1, R2, R3
 $\{R1, R0\} = \{R1, R0\} + (R2 * R3)$

UMULL R4, R5, R6, R7 $\{R5, R4\} = R6 * R7$
 ↓

interpret numbers as (+ve) only

UMLAL R4, R5, R6, R7

$\{R5, R4\} = \{R5, R4\} + (R6 * R7)$

only RSBs exists for conditional updating of flags.

Questions

- 1) Write the operation of following code snippet.

LDR R0,=0x12345678

LDR R1,=0xABCDEF00

UMULL R2,R3,R0,R1

After Execution:

R0 x R1 = 0xC37 9AAA 42D2 0800

R2 = 0x42D2 0800 (Lower 32 bits)

R3 = 0x0C37 9AAA (Upper 32 bits)

$\{R3, R2\} = R0 \times R1$
 ↳ lower 32b
 ↳ upper 32b

56

MGR,ECE,RVCE

- 2) Write the operation of following code snippet.

LDR R0,=0xFFFFFFFF

LDR R1,=0xFFFFFFFF

LDR R3,=0x1234

LDR R2,=0x0000

UMLAL R2,R3,R0,R1

After Execution:

R0xR1 = FFFF FFFE 0000 0001

+

0000 0000 0000 1234
 R3 R2

[R3,R2] = FFFF FFFE 0000 1235

$\{R3, R2\} = \{R3, R2\} + (R0 \times R1)$

Questions

- 3) Write the operation of following code snippet.

LDR R0,=0x1112222(+ve number)

LDR R1,=0x1112222(+ve number)

LDR R3,=0x0000

LDR R2,=0x0000

SMLAL R2,R3,R0,R1

After Execution:

R0 * R1 = 0x12369 D111 0C84(+ve number)

R2 = 0x D111 0C84 (Lower 32 bits)

R3 = 0x0001 2369 (Upper 32 bits)

- Signed Multiplication rules:

- $+A * +B = +AB$
- $+A * -B = -AB$
- $-A * -B = +AB$

(NOTE)!!

57

MGR,ECE,RVCE

Questions

- 1) Write the operation of following code snippet.

LDR R0,=0x81112222(-ve number)

LDR R1,=0x81112222(-ve number)

LDR R3,=0x0000

LDR R2,=0x0000

SMLAL R2,R3,R0,R1

After Execution:

$R0 * R1 = 2\text{'s complement}(0x81112222) *$

$2\text{'s complement}(0x81112222)$

$= 0x7EEEDDDE * 0x7EEEDDDE$

$= 0x3EF0\ 0147\ D111\ 0C84$

$R2 = 0x\ D111\ 0C84$ (Lower 32 bits)

$R3 = 0x\ 3EF0\ 0147$ (Upper 32 bits)

0110 - 2
1101 - 0
+ 110 - E

- 2) Write the operation of following code snippet.

LDR R0,=0x81112222(-ve number)

LDR R1,=0x1112222(+ve number)

LDR R3,=0x0000

LDR R2,=0x0000

SMULL R2,R3,R0,R1

After Execution:

$R0 * R1 = 2\text{'s complement}(0x81112222) *$

$0x1112222$

$= 0x7EEEDDDE * 0x1112222$

$= 0x87\ 6DA7\ 2EEE\ F37C$ (-ve number)

Result = $2\text{'s complement}(0x87\ 6DA7\ 2EEE\ F37C)$

$= 0xFF78\ 9258\ D111\ 0C84$

$R2 = 0x\ D111\ 0C84$ (Lower 32 bits)

$R3 = 0x\ FF78\ 9258$ (Upper 32 bits)

Division: —

UDIV R3,R0,R1 $R3 = R0/R1$

if $R1 = 0$ $R3 = 0$ ($R0/R1$)
[No zero division error]

- Signed Division rules

- + Dividend/ + Divisor = +Q and +R
- + Dividend/ - Divisor = - Q and +R
- - Dividend/ + Divisor = - Q and - R
- - Dividend/ - Divisor = +Q and - R

SDIV R3,R0,R1 $R3 = R0/R1$

SDIV Instruction...

- 1) Identify the operation of following code snippet

LDR R0,=0x81112222(-ve number)

LDR R1,=0x90002222(-ve number)

LDR R3,=0x0000

SDIV R3,R0,R1

After execution:

$R0/R1 = 2\text{'s complement}(0x81112222)/$

$2\text{'s complement}(0x90002222)$

$= 0x7EEEDDDE / 0x6FFF\ DDDE$

$= 0x1$

$R3 = 0x1$

- 2) Identify the operation of following code snippet

LDR R0,=0x71112222(+ve number)

LDR R1,=0x90002222(-ve number)

LDR R3,=0x0000

SDIV R3,R0,R1

After execution:

$R0/R1 = 0x71112222 /$

$2\text{'s complement}(0x90002222)$

$= 0x71112222 / 0x6FFF\ DDDE$

$= 0x1$

$R3 = 2\text{'s complement}(0x1)$

$R3 = 0xFFFFFFFF$

Instruction	Operation
AND Rd, Rn	$Rd = Rd \& Rn$
AND Rd, Rn, #immed	$Rd = Rn \& \#immed$
AND Rd, Rn, Rm	$Rd = Rn \& Rm$
ORR Rd, Rn	$Rd = Rd Rn$
ORR Rd, Rn, #immed	$Rd = Rn \#immed$
ORR Rd, Rn, Rm	$Rd = Rn Rm$
BIC Rd, Rn	$Rd = Rd \& (\sim Rn)$
BIC Rd, Rn, #immed	$Rd = Rn \& (\sim \#immed)$
BIC Rd, Rn, Rm	$Rd = Rn \& (\sim Rm)$

*No flags are updated because suffix 'S' is not there
 * All instructions are conditional

CE, RVCE

→ Bitwise

AND R0, R1 $R0 = R0 \& R1$

AND R2, R3, #0xFF $R2 = R3 \& 0xFF$

keeps only lower 8 bits of R3

AND R4, R5, R6 $R4 = R5 \& R6$

→ Bitwise

ORR R0, R1 $R0 = R0 | R1$

ORR R2, R3, #0x0F $R2 = R3 | 0x0F$
 forces last 4 bits to 1

ORR R4, R5, R6 $R4 = R5 | R6$

→ Bit Clear

BIC R0, R1 $R0 = R0 \& (\sim R1)$

BIC R2, R3, #0x01 $R2 = R3 \& (\sim 0x01)$

clears bit 0

BIC R4, R5, R6 $R4 = R5 \& (\sim R6)$

Instruction	Operation
ORN Rd, Rn, #immed	$Rd = Rn (\sim \#immed)$
ORN Rd, Rn, Rm	$Rd = Rn (\sim Rm)$
EOR Rd, Rn	$Rd = Rd \wedge Rn$
EOR Rd, Rn, #immed	$Rd = Rn \wedge \#immed$
EOR Rd, Rn, Rm	$Rd = Rn \wedge Rm$

*No flags are updated because suffix 'S' is not there
 * All instructions are conditional

→ OR NOT

ORN R0, R1, #0xFF $R0 = R1 | (\sim 0xFF)$

ORN R2, R3, R4 $R2 = R3 | (\sim R4)$

→ Exclusive OR (XOR)

EOR R0, R1 $R0 = R0 \wedge R1$

EOR R2, R3, #0x01 $R2 = R3 \wedge 0x01$

toggles bit 0

EOR R4, R5, R6 $R4 = R5 \wedge R6$

Logical Instructions ...

- Identify the operation & working of following C program statements:

```
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2 | GPIO_PIN_3, GPIO_PIN_RESET);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4 | GPIO_PIN_5 | GPIO_PIN_6 | GPIO_PIN_7, GPIO_PIN_SET);
```

Pin definitions:

```
#define GPIO_PIN_0 ((uint16_t)0x0001)
#define GPIO_PIN_1 ((uint16_t)0x0002)
#define GPIO_PIN_2 ((uint16_t)0x0004)
#define GPIO_PIN_3 ((uint16_t)0x0008)
#define GPIO_PIN_4 ((uint16_t)0x0010)
#define GPIO_PIN_5 ((uint16_t)0x0020)
#define GPIO_PIN_6 ((uint16_t)0x0040)
#define GPIO_PIN_7 ((uint16_t)0x0080)
#define GPIO_PIN_8 ((uint16_t)0x0100)
#define GPIO_PIN_9 ((uint16_t)0x0200)
#define GPIO_PIN_10 ((uint16_t)0x0400)
#define GPIO_PIN_11 ((uint16_t)0x0800)
#define GPIO_PIN_12 ((uint16_t)0x1000)
#define GPIO_PIN_13 ((uint16_t)0x2000)
#define GPIO_PIN_14 ((uint16_t)0x4000)
#define GPIO_PIN_15 ((uint16_t)0x8000)
#define GPIO_PIN_All ((uint16_t)0xFFFF)
```

Combined Value

pins 23 → reset (sets 0v)

14567 → set (sets to high volt)

Note:

→ ORing (forcing bits)

$\{a | 1 = 1\}$
 $\{a | 0 = a\}$

eg: $0x12345678 | 0x0000FFFF$

$= 0x1234FFFF$

use: GPIO_PIN_2 | GPIO_PIN_3

→ ANDing (Masking)

$\{a \& 0 = 0; a \& 1 = a\}$

eg: AND R2, R0, #0x0FFF
 upper bits cleared to 0

→ Toggling (XOR)

$\{a \wedge 1 = \sim a; a \wedge 0 = a\}$

eg: to flip specific bits
 $0x78 \oplus 0x6F = 0x17$

use: LED blinking

Instruction	Operation
ASR Rd, Rn, #immed	$Rd = Rn \gg \text{immed}$ (Arithmetic Shift)
ASR Rd, Rn	$Rd = Rn \gg Rn$
ASR Rd, Rn, Rm	$Rd = Rn \gg Rm$
LSL Rd, Rn, #immed	$Rd = Rn \ll \text{immed}$
LSL Rd, Rn	$Rd = Rn \ll Rn$
LSL Rd, Rn, Rm	$Rd = Rn \ll Rm$
LSR Rd, Rn, #immed	$Rd = Rn \gg \text{immed}$ (Logical Shift)
LSR Rd, Rn	$Rd = Rn \gg Rn$
LSR Rd, Rn, Rm	$Rd = Rn \gg Rm$

* Carry flag is updated if suffix 'S' is there

* All instructions are conditional

MGR

→ Arithmetic Shift Right

ASR R0, R1, #2 $R0 = R1 \ggg 2$

$R1 = 0x9000_2222 \rightarrow R0 = 0x400_0888$
 msb 0001 0010 0010 1110 1000
 0010 0010 1000 8

ASR R2, R3 $R2 = R3 \ggg R3$

ASR R4, R5, R6 $R4 = R5 \ggg R6$

→ Logical Shift Left

LSL R0, R1, #1 $R0 = R1 \ll 1$

LSL R2, R3 $R2 = R2 \ll R3$

LSL R4, R5, R6 $R4 = R5 \ll R6$

→ Logical Shift Right

LSR R0, R1, #1 $R0 = R1 \lll 1$

LSR R2, R3 $R2 = R2 \lll R3$

LSR R4, R5, R6 $R4 = R5 \lll R6$

Instruction	Operation
ROR Rd, Rn	$Rd = Rn$ rotate by Rn
ROR Rd, Rn, Rm	$Rd = Rn$ rotate by Rm
RRX Rd, Rn	RRX provides the value of the contents of a register shifted right one bit. The old carry flag is shifted into bit[31]. If the S suffix is present, the old bit[0] is placed in the carry flag. [C, Rd] = [Rn, C]

* Carry flag is updated if suffix 'S' is there

* All instructions are conditional

MCD

ROR R4, R1 $R4 = R1 \gg R1$

ROR R4, R1, R2 $R4 = R1 \gg R2$

$R1 = 0x9000_2222$

1001 0010 0010
 (A) 1010 (W) 1000 = $R4 = 0x400_0888$
 ROR R4, R1, #2

Instruction	Operation
CMP Rn, Rm	Compare: Calculate $Rn - Rm$. APSR is updated but the result is not stored.
CMP Rn, #immed	Compare: Calculate $Rn - \text{immediate data}$.
CMN Rn, Rm	Compare negative: Calculate $Rn + Rm$. APSR is updated but the result is not stored.
CMN Rn, #immed	Compare negative: Calculate $Rn + \text{immediate data}$. APSR is updated but the result is not stored.
TST Rn, Rm	Test (bitwise AND): Calculate AND result between Rn and Rm. N bit and Z bit in APSR are updated but the AND result is not stored.
TEQ Rn, #immed	Test (bitwise XOR): Calculate XOR result between Rn and immediate data. N bit and Z bit in APSR are updated but the AND result is not stored.

→ Compare calculate

CMP R0, R1 $R0 - R1$; if $R0 = R1 \rightarrow Z = 1$

CMP R2, #10 $R2 - 10$

→ Compare -ve

CMN R4, R5 $R4 + R5 \rightarrow$ update APSR flags
 CMN R6, #5 $R6 + 5$

→ test

TST R0, R1 $R0 \text{ AND } R1$; APSR flags updated
 $= 0 \rightarrow Z = 1$

→ test equivalence

TEQ R3, #0xFF $R3 \text{ XOR } 0xFF$
 $= 0$ (identical) $\Rightarrow Z = 1$

→ Rotate Right w/ extend

RRX R0, R1 $R0 = R0 \ggg R1$

$R1 = 0x9000_2222$; C = 0

→ 01001 0010 0010

C → 00100 (4) (1) 0001 = R0

RRX R0, R1; $R0 = 4800_111$
 C = 0

Shift Instructions...

Identify the difference between following statements:

• LDR R1,=0x90002222
LDR R4,=0x0000
LSL R4, R1,#1

After execution:

R1=0x90002222

R4=0x20004444

No carry flag updated



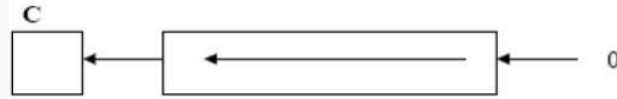
LDR R1,=0x90002222
LDR R4,=0x0000
LSL R4, R1,#1

After execution:

R1=0x90002222

R4=0x20004444

Carry flag=1



70

MGR,ECE,RVCE

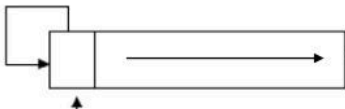
LDR R1,=0x90002222
LDR R4,=0x0000
ASR R4, R1,#2

After execution:

R1=0x90002222

R4=0xE400 0888

No carry flag updated



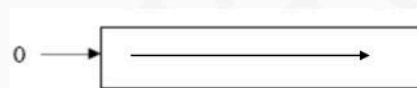
LDR R1,=0x90002222
LDR R4,=0x0000
LSR R4, R1,#1

After execution:

R1=0x90002222

R4=0x2400 0888

No carry flag updated



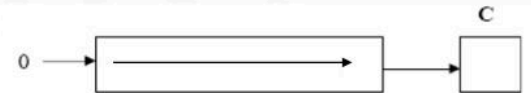
LDR R1,=0x90002222
LDR R4,=0x0000
LSRS R4, R1,#1

After execution:

R1=0x90002222

R4=0x2400 0888

Carry flag=1



Shift Instructions...

Identify the difference between following code snippets:

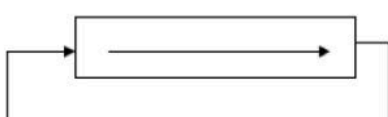
LDR R1,=0x90002222
LDR R4,=0x0000
ROR R4, R1,#2

After execution:

R1=0x90002222

R4=0xA400 0888

No carry flag updated



LDR R1,=0x90002222
LDR R4,=0x0000
RORS R4, R1,#2

After execution:

R1=0x90002222

R4=0xA400 0888

Carry flag=1



LDR R1,=0x90002222
RRX R0, R1

After execution:

R1=0x48001111



72

MGR,ECE,RVCE

Unconditional Branch Instructions

Instruction	Operation
B label B.W label	Branch to label. If a branch range of over +/- 2KB is needed, you might need to specify B.W to use 32-bit version of branch instruction for wider range.
BX Rm	Branch and exchange. Branch to an address value stored in Rm.

Instructions for calling function

Instruction	Operation
BL label	Branch to a labeled address and save the return address in LR
BLX Rm	Branch to an address specified by Rm, save the return address in LR.

B my-loop jumps to mem.
addr identified by label my-loop

BX R5 jumps to addr currently stored in reg. R5.

BL PrintText branches to label PrintText and automatically saves the addr of next inst. into LR (return addr)
(link reg.)

BLX R0 branches to addr specified in R0 and saves return addr in LR

Conditional Branch Instruction

Instruction	Operation
B<cond> <label>	Branch to label if condition is true

Suffixes for Conditional Branches and Conditional Execution

Suffix	Branch Condition	Flags(APSR)
EQ	Equal	Z=1
NE	Not Equal	Z=0
CS/HS	Carry Set/Unsigned Higher or same	C=1
CC/LO	Carry clear / unsigned lower	C=0
MI	Minus	N=1
PL	Plus	N=0

branches if
equal
not equal
carry
no carry
-ve
+ve / 0

eg:

CMP R0, #10 ; compare R0 w/ 10
calcs R0-10

BEQ equal-label ; if R0==10 (Z=1)
jumps to equal-label

BNE diff-label ; if R0!=10 (Z=0)
jumps to diff-label

Conditional Branch Instruction

Suffix	Branch Condition	Flags(APSR)
VS	Overflow	V=1
VC	No Overflow	V=0
GE	Signed greater than or equal	N==V or Z=1
LE	Signed less than or equal	N!=V or Z=1
GT	Greater Than	N==V
LT	Less Than	N!=V

